Guia completo de GitLab Flow

> Workflow com **main** + branches de ambiente perenes (`staging` , `production`), regra *upstream first*, e o fluxo de cada cenário prático — feature, bugfix, fix, hotfix, release, revert — com comandos git passo a passo e diagramas.

1. Conceitos-chave do GitLab Flow

GitLab Flow é um híbrido pragmático: mantém a simplicidade de "uma main" do GitHub Flow, mas adiciona **branches de ambiente perenes** para garantir janelas de validação antes de produção. Três regras definem todo o resto.

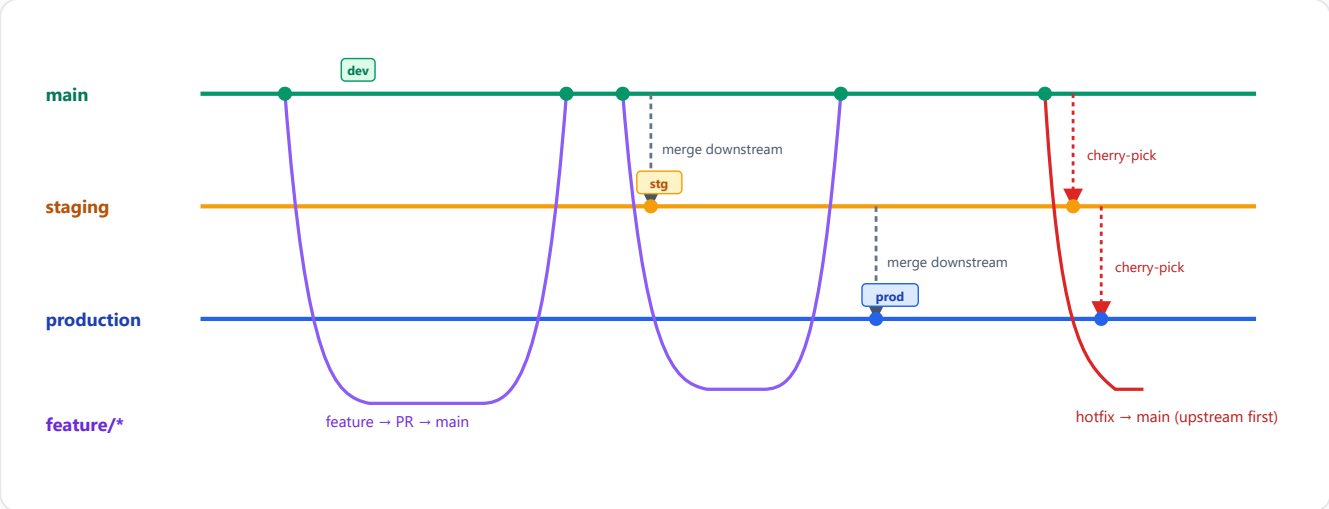
As 3 regras fundamentais

1. **Upstream first**
   : toda mudança (feature, fix, hotfix) entra em `main` *antes* de qualquer branch de ambiente. Nenhuma exceção.
2. **Merges fluem sempre para baixo**
   : `main → staging → production` . Nunca o contrário. Branches de ambiente não recebem commits diretos — só merges ou cherry-picks vindos de cima.
3. **Cada branch de ambiente = um ambiente real**
   . Push/merge naquela branch dispara deploy automaticamente no ambiente correspondente. Não existe mapeamento indireto.

Por que a regra *upstream first*?

Se você corrigir um bug direto na branch `production` e esquecer de levar a correção para `main` , no próximo release o bug volta — porque main vai sobrescrever production com código antigo. *Upstream first* elimina essa classe de erro por construção: a correção *precisa* passar por main.

Visão geral do fluxo



main (integração) staging production feature/* hotfix (upstream then cherry-pick)

2. Estrutura de branches

Branches permanentes

BRANCH	REPRESENTA	COMO RECEBE CÓDIGO	DISPARA DEPLOY EM
main	Integração contínua — o que o time está construindo. Sempre deployável em dev.	Só via PR de `feature/*`, `bugfix/*`, `hotfix/*`	**DEV**
staging	Candidato a release, em validação por QA.	Merge de `main` (promoção) ou cherry-pick de `main` (hotfix)	**STG**
production	Exatamente o que está em produção. Toda entrada vira tag SemVer.	Merge de `staging` ou cherry-pick de `main` (hotfix)	**PROD**

Nomes alternativos

Alguns times preferem `pre-production` no lugar de `staging`. Pode usar qualquer um — o importante é que o nome seja estável e corresponda 1:1 a um ambiente real. Evite renomear depois que o fluxo estiver em uso.

Branches de trabalho (temporárias)

PREFIXO	NASCE DE	VOLTA PARA (PR)	USO
feature/	main	main	Nova funcionalidade.
bugfix/	main	main	Correção de bug que *não* está em produção ainda (encontrado em dev ou staging).
hotfix/	main	main, + cherry-pick para `staging` e `production`	Correção **urgente** de bug que *está* em produção.
fix/	main	main	Sinônimo informal de bugfix. Alguns times usam; outros padronizam em `bugfix/`.
chore/	main	main	Ferramental, deps, CI — não entregam comportamento novo.

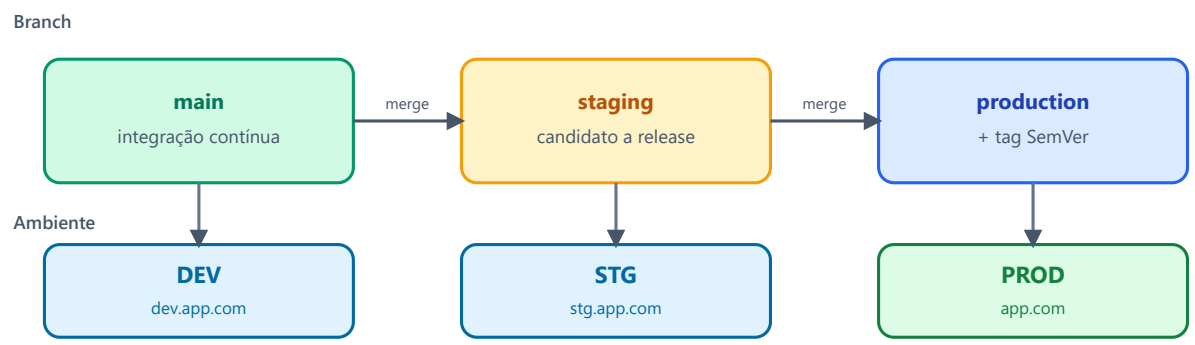
A diferença prática entre fix, bugfix e hotfix

Os três são tecnicamente correções de bug. A diferença é *onde o bug foi descoberto*:

- fix/ / bugfix/ — bug visto em **dev ou staging**. Ainda não chegou em produção. Segue o fluxo normal: `main → staging → production` no próximo release. Sem pressa, sem cherry-pick.
- hotfix/ — bug **em produção**. Precisa chegar em prod agora, pulando a espera pelo próximo release. Por isso o cherry-pick para `staging` e `production` após o merge em main.

Esta é *a* distinção crítica. O processo de código é quase idêntico; o que muda é a urgência e os cherry-picks downstream.

3. Branches × ambientes



AMBIENTE	BRANCH	COMO CÓDIGO CHEGA LÁ	QUEM USA
DEV	`main`	Merge de PR de feature/bugfix/hotfix	Devs integrando features; QA explorando
STG	`staging`	Merge `main → staging` (promoção planejada) OU cherry-pick de `main` (hotfix)	QA fazendo validação de release; stakeholders
PROD	`production`	Merge `staging → production` (promoção pós-QA) OU cherry-pick de `main` (hotfix) + tag SemVer	Usuários reais

Proteções recomendadas (GitHub/GitLab settings)

- `main` : PR obrigatório, 1+ aprovação, CI verde, sem push direto.
- `staging` : PR obrigatório, 1 aprovação (ou auto se vier de main via promoção), sem push direto.
- `production` : PR obrigatório, 2 aprovações, CI verde, sem push direto, **sem force-push**

  .

4. Nomenclatura & commits

Nomenclatura de branches

- **Minúsculas com hífen**. Prefixo do ticket pra rastreabilidade.
- `feature/PROJ-231-login-oauth`
- `bugfix/PROJ-245-frete-sp`
- `hotfix/PROJ-301-calculo-impostos`
- `fix/PROJ-250-label-botao` (informal, se o time preferir)
- `chore/atualiza-axios`

Conventional Commits

Mesmo padrão usado no Git Flow:

```
<tipo>(<escopo>): <descrição curta em minúsculas>

<corpo opcional — o "porquê">

Refs: PROJ-231
```

Tipos frequentes: `feat`, `fix`, `refactor`, `perf`, `test`, `docs`, `chore`, `build`. Para breaking change, use `feat!:` ou `fix!:` e rodapé `BREAKING CHANGE:`.

Tags de versão

Tags SemVer (`v1.4.0`) são aplicadas **só em** `production`, no commit de merge que promoveu staging para production. A tag dispara o deploy ou é apenas anotação, dependendo do pipeline.

5. Cenários de dia-a-dia

Cada cenário abaixo cobre o fluxo completo com os comandos git. São os 7 casos que o time vai encontrar no 99% do tempo.

5.1 Feature nova

Situação: desenvolver uma funcionalidade nova que vai entrar no próximo release.

main → feature/PROJ-231-login-oauth → PR → main →

(fica em main até o próximo release downstream)

1 Branch a partir de main atualizada

```
copiar
git checkout main
git pull --rebase origin main
git checkout -b feature/PROJ-231-login-oauth
```

2 Trabalhe em commits pequenos e coerentes

```
copiar
git add .
git commit -m "feat(auth): adiciona provedor OAuth Google"

# publique a branch cedo pra habilitar preview env + review contínuo
git push -u origin feature/PROJ-231-login-oauth
```

3 Mantenha a branch atualizada com main

```
copiar
git fetch origin
git rebase origin/main
git push --force-with-lease
```

4 Abra o PR contra `main`, com CI verde

Após aprovação, faça **Squash and merge** via UI — 1 commit conventional por feature em `main`.

5 Limpeza

```
copiar
git checkout main
git pull --rebase origin main
git branch -d feature/PROJ-231-login-oauth
git push origin --delete feature/PROJ-231-login-oauth
```

> A feature agora está em `main` e rodando em **DEV**. Ela sobe para staging quando houver promoção agendada (seção 5.5).

5.2 Bugfix — bug encontrado em dev

Situação: um bug foi detectado em DEV (ou durante revisão). O bug **ainda não está em produção**. Sem pressa.

main → bugfix/PROJ-245-frete-sp → PR → main

Processo idêntico ao de feature, só muda o prefixo da branch e o tipo do commit:

```
copiar
git checkout main
git pull --rebase origin main
git checkout -b bugfix/PROJ-245-frete-sp

# aplica correção
git add .
git commit -m "fix(checkout): corrige cálculo de frete para SP"
git push -u origin bugfix/PROJ-245-frete-sp

# Abre PR para main, aprova, squash-and-merge
```

Por que não precisa cherry-pick?

O bug está *só* em `main`, porque ainda não foi promovido para `staging` ou `production`. Então corrigir em `main` resolve tudo — a próxima promoção downstream já vai levar o fix junto.

5.3 Fix — bug encontrado em staging

Situação: QA encontrou um bug em STG durante validação. O bug está em `main` e em `staging`, mas **ainda não em produção**.

main → fix/PROJ-260-timeout-gateway → PR → main → cherry-pick → staging

1 Corrige upstream primeiro (em main)

```
copiar
git checkout main
git pull --rebase origin main
```

```
git checkout -b fix/PROJ-260-timeout-gateway

# aplica correção
git commit -m "fix(checkout): trata timeout do gateway de pagamento"
git push -u origin fix/PROJ-260-timeout-gateway

# PR → main, aprovação, squash-and-merge
```

2 Anote o SHA do merge em main

```
copiar
git checkout main
git pull --rebase origin main
git log --oneline -1
# copie o SHA do commit que acabou de entrar
```

3 Cherry-pick para staging

```
copiar
git checkout staging
git pull --rebase origin staging
git cherry-pick <SHA-do-fix-em-main>
git push origin staging
# → deploy automático em STG. QA revalida.
```

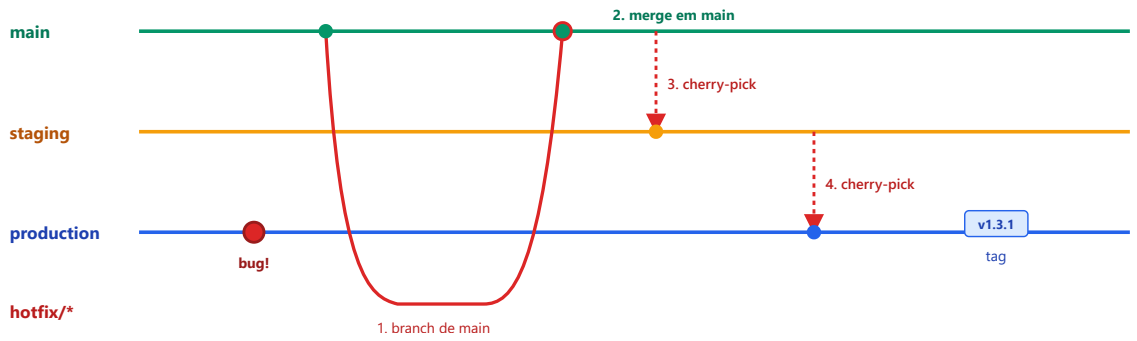
Por que não promover main inteira para staging?

Porque `main` pode ter features incompletas ou em teste que ainda não estão prontas para o release atual. Cherry-pick é cirúrgico: leva *só* o fix, preservando o escopo da release em staging.

5.4 Hotfix — bug crítico em produção

Situação: cliente está reportando bug **em produção**. Precisa ser consertado agora, sem esperar o próximo release. O bug está em `main`, `staging` e `production`.

main → hotfix/PROJ-301 → PR → main → cherry-pick → staging → cherry-pick → production



Upstream first: bug em prod → fix em main → cherry-pick descendo até prod → tag de patch.

1 Crie a branch de hotfix a partir de `main`

```
git checkout main
git pull --rebase origin main
git checkout -b hotfix/PROJ-301-calculo-impostos

# aplica correção mínima possível
git add .
git commit -m "fix(billing): corrige cálculo de impostos em UF externa"
git push -u origin hotfix/PROJ-301-calculo-impostos
```

> **Não tem atalho**
>
> Mesmo que a branch `production` seja a que está quebrada, **não** parta de `production` para o hotfix. Parta de `main`. Caso contrário, você vai ter que fazer o fix "subir" depois, e é aí que erros acontecem. Upstream first, sempre.

2 PR do hotfix → main, com review acelerado

Mesmo sob pressão, o hotfix **passa por PR**: CI verde e 1+ aprovação. Isso evita que o remédio seja pior que o bug. Após merge, anote o SHA:

```
git checkout main
git pull --rebase origin main
git log --oneline -1
# anote o SHA. Exemplo: 9a8b7c6
```

3 Cherry-pick para `staging` e revalidação rápida

```
copiar
git checkout staging
git pull --rebase origin staging
git cherry-pick 9a8b7c6
git push origin staging
# → deploy em STG. QA faz smoke em ~10 min.
```

4 Cherry-pick para `production` + tag de patch

```
copiar
git checkout production
git pull --rebase origin production
git cherry-pick 9a8b7c6

# bump de patch version (v1.3.0 → v1.3.1)
npm version 1.3.1 --no-git-tag-version
git add package.json package-lock.json
git commit -m "chore(release): bump para 1.3.1"

git tag -a v1.3.1 -m "Hotfix 1.3.1 — cálculo de impostos"
git push origin production --tags
# → deploy em PROD disparado pela tag.
```

5 Limpeza

```
copiar
git branch -d hotfix/PROJ-301-calculo-impostos
git push origin --delete hotfix/PROJ-301-calculo-impostos
```

Conflito no cherry-pick?

Pode acontecer se `staging` ou `production` já divergiram demais de `main` desde o último release. Resolva o conflito, `git add`, `git cherry-pick --continue`, teste localmente antes de empurrar. Se o conflito for grande, é sinal de que as env branches estão envelhecidas — faça uma promoção completa assim que possível.

5.5 Release — promover main para staging e production

Situação: sprint terminou, main está com features prontas, é hora de fazer o release planejado do ciclo (semanal/quinzenal).

main → merge → staging → (QA valida) → merge → production → tag v1.4.0

1 Promoção main → staging

```
# via CLI (opcional — o normal é fazer via PR na UI)
git checkout staging
git pull --rebase origin staging
git merge --no-ff origin/main -m "chore(release): promove main → staging para 1.4.0"
git push origin staging
# → deploy em STG. QA valida (E2E completos, smoke, regressão).
```

Preferível via PR: abra um PR `main → staging` chamado "Release 1.4.0 candidate". Isso gera um checkpoint revisável, com changelog automático dos commits que estão entrando.

2 Se QA encontrar bug em staging

Siga o cenário 5.3 (Fix em staging): corrigir em `main` primeiro, depois cherry-pick para `staging`. **Nunca** commit direto em `staging`.

3 Promoção staging → production

```
git checkout production
git pull --rebase origin production
git merge --no-ff origin/staging -m "chore(release): 1.4.0 — staging → production"

# bump de versão + tag
npm version 1.4.0 --no-git-tag-version
git add package.json package-lock.json
git commit -m "chore(release): bump para 1.4.0"
git tag -a v1.4.0 -m "Release 1.4.0"

git push origin production --tags
# → deploy em PROD (com aprovação humana no pipeline, idealmente).
```

Ritmo recomendado

Com cadência semanal/quinzenal, rotule um dia fixo pra promoção (ex: *terça manhã → staging, quinta tarde → production*). Ter previsibilidade reduz ansiedade e simplifica a comunicação com stakeholders.

5.6 Revert — desfazer uma mudança que chegou em produção

Situação: uma feature já em produção está causando problemas (não um bug simples, mas um comportamento errado de design). Melhor desativar e reavaliar.

main → revert branch → PR → main → cherry-pick → staging → cherry-pick → production

Revert segue exatamente o fluxo de hotfix: upstream first.

copiar

```
# 1) Revert em main
git checkout main
git pull --rebase origin main
git checkout -b revert/PROJ-350-feature-oauth

# descobrir o SHA do merge-commit da feature original
git log --oneline --merges | head -20

# Revert (se foi squash-merge, é um commit normal; use -m 1 apenas se foi merge commit)
git revert <sha-do-commit-original>
# OU, se foi um merge commit:
# git revert -m 1 <sha-do-merge>

git commit --amend -m "revert: \"feat(auth): adiciona provedor OAuth Google\""
git push -u origin revert/PROJ-350-feature-oauth

# PR → main, merge

# 2) Cherry-pick para staging e production, igual hotfix
git checkout staging
git cherry-pick <sha-do-revert-em-main>
git push origin staging

git checkout production
git cherry-pick <sha-do-revert-em-main>
npm version patch --no-git-tag-version   # bump de patch
```

```
git add package.json && git commit -m "chore(release): bump patch"
git tag -a v1.4.1 -m "Revert OAuth provider"
git push origin production --tags
```

> **Revert vs. fix**
>
> Revert é quando a solução é "desfazer". Fix é quando a solução é "consertar". Se dá pra consertar rápido, prefira fix (mantém o valor entregue). Revert é o movimento de último recurso, mas é valioso — tira pressão do time.

5.7 (Opcional) Múltiplas versões em produção — release branches

Situação: produto tem versões diferentes rodando simultaneamente em clientes diferentes (ex: on-prem, enterprise vs. cloud). Precisa manter múltiplas *versões estáveis*.

Para 99% dos SaaS, esse caso **não** se aplica — só há uma versão em prod. Mas se aplica, GitLab Flow estende naturalmente com **release branches** imutáveis por versão.

- Ao fazer release, crie `release-1.4` (em vez de simplesmente mergear em `production`).
- Essa branch vira a "produção da versão 1.4". Vira perene (enquanto a versão for suportada).
- Bugs em 1.4 viram *upstream-first* em `main`, depois cherry-pick para `release-1.4` (e outras versões afetadas).

copiar

```
# Corte de release
git checkout main
git pull --rebase origin main
git checkout -b release-1.4
git push -u origin release-1.4
git tag -a v1.4.0 -m "Release 1.4.0"
git push origin --tags

# Bug em 1.4.0 depois de algumas semanas:
# 1) upstream-first em main
# 2) cherry-pick para release-1.4
git checkout release-1.4
git cherry-pick <sha-em-main>
```

```
git tag v1.4.1
git push origin release-1.4 --tags
```

Não use isso se você não precisa

Release branches multiplicam o trabalho de cherry-pick (uma por versão suportada). Adote só se o negócio realmente exige múltiplas versões em paralelo em prod.

6. Pull Requests & code review

Estratégia de merge por tipo de PR

PR	ESTRATÉGIA	MOTIVO
`feature/* → main`	**Squash and merge**	1 commit conventional por feature, histórico de main limpo.
`bugfix/* → main`	**Squash and merge**	Idem.
`hotfix/* → main`	**Squash and merge**	1 commit = facilita cherry-pick downstream.
`main → staging` (promoção)	**Merge commit (--no-ff)**	Preserva o "ponto" da promoção. Visível em `git log --graph`.
`staging → production` (promoção)	**Merge commit (--no-ff)**	Idem. + tag SemVer.

Por que squash nos PRs de trabalho importa muito aqui

O PR `hotfix/* → main` ideal é 1 único commit. Isso simplifica os cherry-picks para `staging` e `production` — você cherry-picka 1 SHA e pronto. Se forem N commits, você teria que cherry-pickar N, ou fazer rebase interativo. Squash-and-merge elimina esse atrito.

Template de PR

```
## O que muda
<1-3 linhas>

## Por quê
<contexto, link do ticket>

## Tipo
- [ ] Feature
- [ ] Bugfix (bug não está em prod)
- [ ] Hotfix (bug está em prod — precisa cherry-pick para staging e production)
- [ ] Chore

## Plano de cherry-pick (se hotfix)
- [ ] Cherry-pick para staging: SHA = ___
- [ ] Cherry-pick para production: SHA = ___
- [ ] Tag de patch version em production

## Como validar
- [ ] ...

## Riscos & rollback
<...>

Refs: PROJ-231
```

copiar

7. CI/CD por branch

BRANCH / GATILHO	PIPELINE	AMBIENTE
PR contra `main`	Lint + type-check + build + unit + E2E smoke (críticos)	Preview env (efêmero)
Push em `main`	Tudo acima + E2E completos + deploy	**DEV**
Push em `staging` (via merge ou cherry-pick)	Build + E2E completos + regressão + deploy	**STG**
Push em `production`	Build + smoke E2E pós-deploy + deploy *com aprovação humana*	**PROD**

Tag `vX.Y.Z`	Gera release notes, atualiza changelog, notifica time	Sem deploy (deploy já ocorreu no push em production)

> **Cuidado com deploy em cherry-pick**
>
> Se seu pipeline dispara deploy em qualquer push para `staging` ou `production`, o cherry-pick de hotfix já dispara deploy. Isso é o comportamento desejado, mas o time precisa saber — não é "só um git command", é um deploy em produção.

8. Armadilhas comuns (e como evitar)

Commit direto em `staging` ou `production`

É *a* violação mais comum. Parece atalho, mas quebra a garantia de "upstream first" e na próxima promoção o commit some (ou reaparece como conflito). **Proteger as branches no Git provider** e exigir PR é a defesa mais eficaz.

Esquecer o cherry-pick depois do hotfix

Você mergeou o hotfix em main, mas esqueceu de cherry-pickar para staging e production. Resultado: prod continua quebrado. **Checklist no PR de hotfix** (com os SHAs) mata esse problema.

Cherry-pick de vários commits de uma vez

Se o hotfix ficou com 5 commits em main, cherry-pickar os 5 em ordem é frágil. **Squash no PR de hotfix** transforma o trabalho em 1 SHA — cherry-pick vira trivial.

Staging envelhece

Se ninguém faz promoção `main → staging` por semanas, o cherry-pick de hotfixes acumulados começa a dar conflito. **Faça promoção em cadência fixa**, mesmo que o lote esteja pequeno.

Promover main → staging com feature incompleta

Usar "fix" e "bugfix" e "hotfix" inconsistentemente

Se main tem feature meio-pronta, a promoção leva ela junto. Soluções: (a) feature flags para esconder o que não está pronto; (b) segurar o merge do PR da feature até estar pronto.

Time confunde: "é urgente ou não?". **Defina no CONTRIBUTING.md** e no template de PR: "hotfix = bug em prod". Usar só `bugfix/` e `hotfix/` (pulando `fix/`) é uma simplificação válida.

9. FAQ

Preciso de uma branch `develop`?

Não. `main` no GitLab Flow exerce o papel que `develop` tinha no Git Flow. A branch `production` substitui `main` do Git Flow como "espelho do que está em prod". Essa economia de uma branch permanente é uma das vantagens.

Como lidar com migrações de banco de dados em hotfix?

Regra: hotfix **não** muda schema. Se você precisa mudar schema, é um release normal (promoção completa), não um hotfix. Migrations destrutivas + cherry-pick é fórmula de desastre.

E se dois hotfixes acontecerem em paralelo?

Fluxo é o mesmo, cada um na sua branch de main. O segundo cherry-pick pode dar conflito no primeiro — resolva, teste, empurre. Em equipe, coordene no canal de incidente pra não cherry-pickar em cima do outro ao mesmo tempo.

Quando usar `merge` vs `cherry-pick` para descer mudança em env branch?

Merge quando quer *todas* as mudanças de main desde o último ponto — é promoção de release. **Cherry-pick** quando quer levar *só* um commit específico — é hotfix ou fix cirúrgico. Nunca misture: ou promove tudo, ou cherry-picka um commit.

O `staging` pode ficar atrás de `production`?

Sim, temporariamente, se um hotfix foi cherry-pickado direto de main para production "pulando" staging (em emergências extremas). Mas você *deve* retroativamente cherry-pickar para staging também, para manter a invariante "production é subset de staging é subset de main". Senão, a próxima promoção main → staging vai re-introduzir o bug.

Como faço rollback em produção?

Duas opções: (1) redeploy da tag anterior (`v1.3.9`), se o pipeline suporta; (2) revert via git seguindo o cenário 5.6. A opção 1 é mais rápida; a 2 é mais "oficial" (deixa rastro). Para incidentes graves, faça 1 primeiro (resolver agora) e depois 2 (consolidar).

Uso SemVer mesmo sem múltiplas versões em produção?

Sim, recomendado. Mesmo com uma versão em prod, SemVer dá significado às mudanças: MINOR = feature, PATCH = fix/hotfix, MAJOR = breaking change. Ajuda time, stakeholders, e release notes.

Posso adotar GitLab Flow vindo de Git Flow?

Sim, e é relativamente suave: (a) renomeie `develop → main`; (b) crie `staging` e `production` a partir de main; (c) pare de criar `release/*` e `hotfix/*` tradicionais — troque o fluxo por promoções + cherry-picks; (d) atualize proteções de branch. Um sprint pra estabilizar o novo fluxo é realista.

10. Cheatsheet

copiar

```
# ===== FEATURE / BUGFIX (bug não em prod) =====
git checkout main && git pull --rebase origin main
git checkout -b feature/PROJ-231-slug   # ou bugfix/PROJ-245-slug
# ... commits ...
```

```
git push -u origin feature/PROJ-231-slug
# PR → main, squash-and-merge

# ===== FIX (bug em staging) =====
git checkout main && git pull --rebase origin main
git checkout -b fix/PROJ-260-slug
git commit -m "fix(escopo): ..."
git push -u origin fix/PROJ-260-slug
# PR → main, squash-and-merge, anote SHA
git checkout staging && git pull --rebase origin staging
git cherry-pick <SHA>
git push origin staging

# ===== HOTFIX (bug em prod) =====
git checkout main && git pull --rebase origin main
git checkout -b hotfix/PROJ-301-slug
git commit -m "fix(escopo): ..."
git push -u origin hotfix/PROJ-301-slug
# PR → main, squash-and-merge, anote SHA
git checkout staging && git cherry-pick <SHA> && git push origin staging
git checkout production && git cherry-pick <SHA>
npm version patch --no-git-tag-version
git add package.json && git commit -m "chore(release): bump patch"
git tag -a vX.Y.Z -m "Hotfix X.Y.Z"
git push origin production --tags

# ===== RELEASE NORMAL =====
# PR main → staging (promoção), merge-commit
# QA valida em STG
# PR staging → production (promoção), merge-commit
git checkout production && git pull --rebase origin production
npm version 1.4.0 --no-git-tag-version
git add package.json && git commit -m "chore(release): bump para 1.4.0"
git tag -a v1.4.0 -m "Release 1.4.0"
git push origin production --tags

# ===== REVERT EM PROD =====
git checkout main && git pull --rebase origin main
git checkout -b revert/PROJ-350-slug
git revert <SHA-original>
# PR → main, merge, anote SHA do revert
git checkout staging && git cherry-pick <SHA-revert> && git push
git checkout production && git cherry-pick <SHA-revert>
npm version patch --no-git-tag-version
```

```
git tag -a vX.Y.Z -m "Revert ..."
git push origin production --tags
```

Resumo de uma linha

Tudo começa em main. Se o bug está em prod, cherry-pick desce. Se é release planejado, merge desce. Env branches são só espelhos do que main já sancionou.